

Lecture 3
Embedding and Vector Representation for Language Modeling

MAI554: Deep Learning for NLP

Prof. Anis Koubaa

Professor

College of Engineering
Alfaisal University

akoubaa@alfaisal.edu

2D Vectors

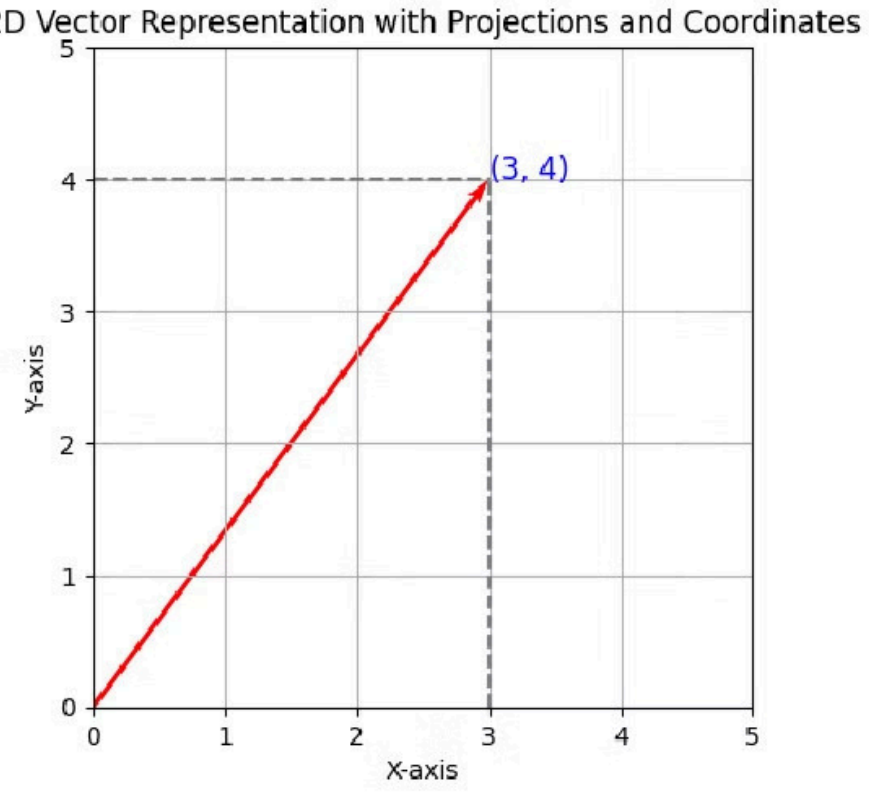
- **Representing Vectors Using 1D Arrays**:

```python
import numpy as np
# Example of initializing a feature vector in NumPy
feature_vector = np.array([5, 3, 1, 8])
```

- **Initializing Vectors**:

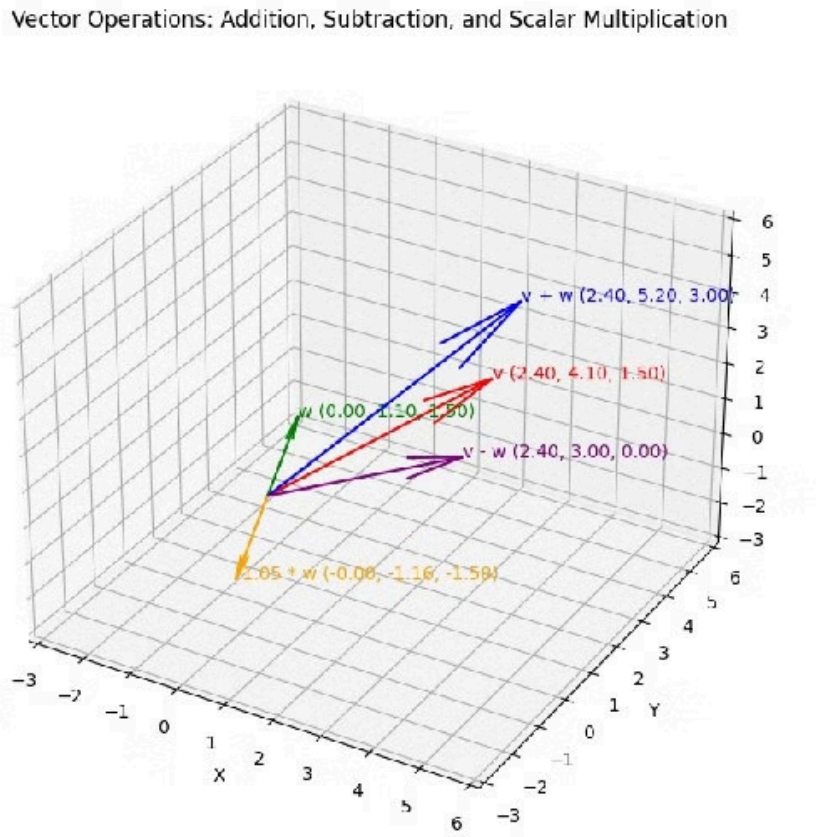
```python
# Creating zero and one vectors commonly used in initialization
zero_vector = np.zeros(4)  # Vector of zeros
one_vector = np.ones(4)    # Vector of ones
```

```
# Define a 2D and a 3D vector
vector_2d = np.array([3, 4])
vector_3d = np.array([3, 4, 5])
```



Vector Operations

- **Operations in Linear Algebra:**
  - **Addition and Subtraction of Vectors:**
    - $\mathbf{v} = [2, 4, 6]$
    - $\mathbf{w} = [1, 0, 1]$
    - **Addition:** $\mathbf{v} + \mathbf{w} = [2+1, 4+0, 6+1] = [3, 4, 7]$
    - **Subtraction:** $\mathbf{v} - \mathbf{w} = [2-1, 4-0, 6-1] = [1, 4, 5]$
  - **Scalar Multiplication:**
    - Scalar $c = 3$
    - $c \cdot \mathbf{v} = 3 \cdot [2, 4, 6] = [6, 12, 18]$



Vector Operations

- Operations in Linear Algebra:
  - Addition and Subtraction of Vectors:
    - $\mathbf{v} = [2, 4, 6]$
    - $\mathbf{w} = [1, 0, 1]$
    - Addition: $\mathbf{v} + \mathbf{w} = [2 + 1, 4 + 0, 6 + 1] = [3, 4, 7]$
    - Subtraction: $\mathbf{v} - \mathbf{w} = [2 - 1, 4 - 0, 6 - 1] = [1, 4, 5]$
  - Scalar Multiplication:
    - Scalar $c = 3$
    - $c \cdot \mathbf{v} = 3 \cdot [2, 4, 6] = [6, 12, 18]$

NumPy Implementation:

- Performing Vector Addition and Subtraction:

```python
import numpy as np
v = np.array([2, 4, 6])
w = np.array([1, 0, 1])
addition = v + w  # Vector addition
subtraction = v - w  # Vector subtraction
```

- Scalar Multiplication:

```python
scalar = 3
scaled_vector = v * scalar  # Scalar multiplication
```

3D Vector Operations Sum | Subtraction

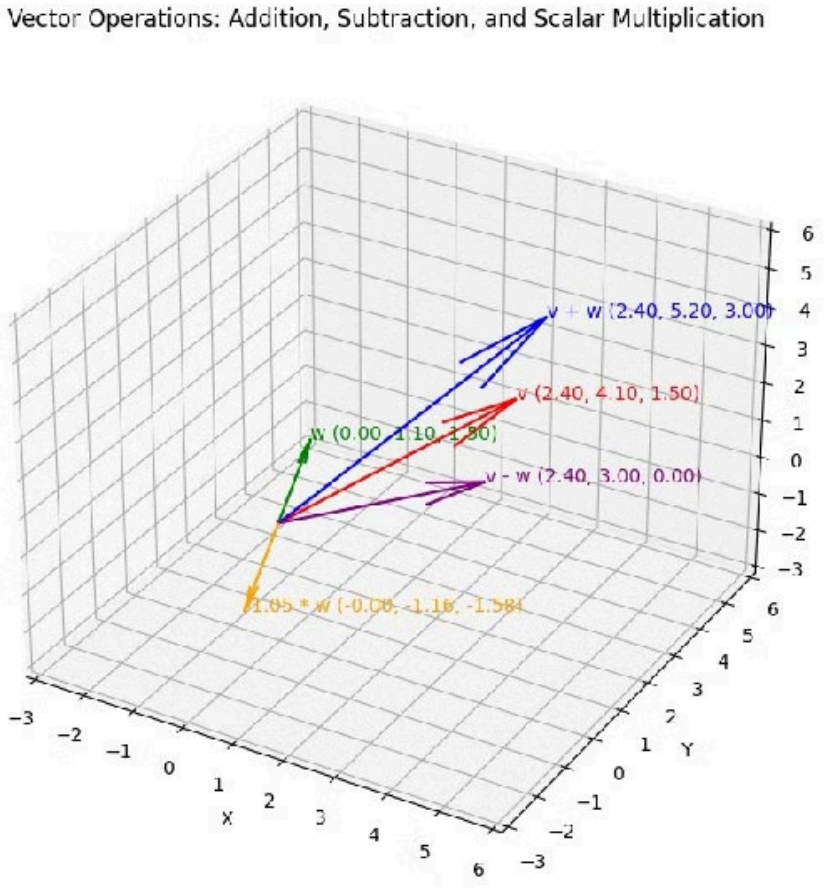
NumPy Implementation:

- **Performing Vector Addition and Subtraction:**

```python
import numpy as np
v = np.array([2, 4, 6])
w = np.array([1, 0, 1])
addition = v + w  # Vector addition
subtraction = v - w  # Vector subtraction
```

- **Scalar Multiplication:**

```python
scalar = 3
scaled_vector = v * scalar  # Scalar multiplication
```



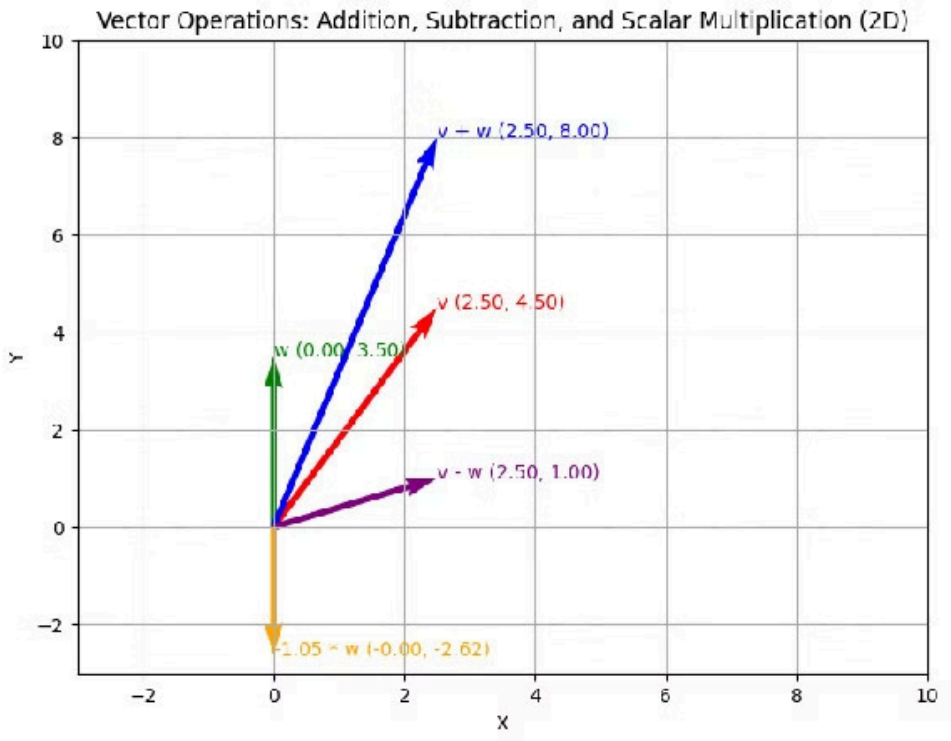
2D Vector Operations - Sum | Subtraction

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the 2D vectors v and w
v = np.array([2.5, 4.5])
w = np.array([0.0, 3.5])

# Vector operations
addition = v + w
subtraction = v - w
c = -0.75
scalar_multiplication = c * w

```



Vectors in Computer Vision

Vectors in Computer Vision:

- **Image Representation:**
  - Images in computer vision are represented as vectors or matrices of pixel values.
  - For example, a grayscale image can be represented as a 2D array where each element corresponds to the intensity of a pixel.
  - **Math Example:** A 2×2 grayscale image might be represented in NumPy as:

```python
import numpy as np
image = np.array([[0, 255], [255, 0]])  # 0 = Black, 255 = White
```

  - This simple array shows how images are treated as numerical data, which AI models can process to recognize patterns or features.

```
Image shape: (275, 183, 3)
Image array:
[[[255 255 255]
  [255 255 255]
  [255 255 255]
  ...
  [255 255 255]
  [255 255 255]
  [255 255 255]]

 [[255 255 255]
  [255 255 255]
  [255 255 255]
  ...
  [255 255 255]
  [255 255 255]
  [255 255 255]]

 [[255 255 255]
  [255 255 255]
  [255 255 255]
  ...
  [255 255 255]
  [255 255 255]
  [255 255 255]]

 ...
```



Vectors in NLP

Vectors in Natural Language Processing (NLP):

- **Word Representation**:
  - Words in NLP are represented as vectors in a high-dimensional space (word embeddings).
  - These vectors capture semantic meanings where words with similar meanings are closer in the vector space.
  - **Math Example**: Representing the word "king" as a vector might be abstractly visualized as:

```python
king = np.array([0.2, -0.4, 0.7, 0.3])  # Hypothetical vector coord
```

  - This vector could be part of a model trained to perform operations like finding synonyms or analyzing text sentiment.

```python
import numpy as np

# Let's assume we have 3 words and we're embedding them in a 5-dimensional vecto
word_to_embedding = {
    'apple': np.array([0.1, 0.3, 0.5, 0.7, 0.9]),
    'banana': np.array([0.2, 0.4, 0.6, 0.8, 1.0]),
    'cherry': np.array([0.3, 0.5, 0.7, 0.9, 1.1])
}

# Let's select a word to embed
word = 'apple'

# Get the word's embedding vector
embedding_vector = word_to_embedding[word]

# Display the embedding vector
print(f"Embedding vector for the word '{word}':")
print(embedding_vector)

# You can perform operations on these vectors, such as calculating similarity
# For example, calculating the dot product between "apple" and "banana"
similarity = np.dot(word_to_embedding['apple'], word_to_embedding['banana'])

print(f"\nSimilarity (dot product) between 'apple' and 'banana': {similarity}")
```

```
Embedding vector for the word 'apple':
[0.1 0.3 0.5 0.7 0.9]

Similarity (dot product) between 'apple' and 'banana': 1.9
```

Application of Vectors in NLP

```python
import spacy
import numpy as np

# Load the small English model in spaCy
nlp = spacy.load("en_core_web_sm")

# Choose a word and convert it to a vector using spaCy
word = "apple"
doc = nlp(word)
embedding_vector = doc.vector

# Convert the spaCy vector to a NumPy array
embedding_vector = np.array(embedding_vector)

# Print the embedding vector
print(f"Embedding vector for the word '{word}':")
print(embedding_vector)

#Print the Shape of the vector
print(f"Shape of the embedding vector for the word '{word}' is {embedding_vector.shape}:")

```

```
Embedding vector for the word 'apple':
[-1.2599487  -0.87038326 -1.0834986   0.5798379   0.03857595 -0.02588724
  0.9775548   0.31093895  0.19412561 -0.8862178   0.43808204 -1.8498268
 -0.30574286  0.5693637   0.42844394  0.69174224 -0.7197368  -1.2614795
  0.83457553  0.14667332  0.12171662  0.48029226  0.50147873 -0.4299112
  0.5533447   0.8749714   0.71914     0.5731143  -0.5064311   0.38493997
 -0.31778833  0.18084693  0.5162936  -0.08233826  0.1870515  -1.3773322
  1.009095   -0.10771251  1.6994228   0.70603184 -0.8166558   0.57896584
 -0.5232718   0.7045958  -0.46308953 -0.37629813 -0.38788998  0.1730735
 -0.05550597 -0.17245518  0.62919456  0.87473     0.60047954 -0.27686393
  0.8524152  -0.28676936  0.9972549  -0.71060055  0.11830124 -0.37214422
 -1.3039289  -0.02281845  0.4063236  -0.43118405  0.9401908  -0.02761412
 -0.39026427 -0.29733896  0.78710043 -0.34422576  0.11906591  0.8003473
  1.4978364  -0.38792044 -0.5264353  -0.38889915 -0.28553864 -0.22295864
  0.8420893  -0.79365766 -0.0956156  -0.96640915 -1.1665895  -1.1019065
 -0.582306    0.765056    0.3859367  -0.31349194 -1.237845    0.11333084
 -1.6053262   0.14791119  0.6127024   1.0456864   0.0747115   0.6120273 ]
Shape of the embedding vector for the word 'apple' is (96,):
```

Explanation:

1. We use the `en_core_web_sm` model from spaCy, which is a small pre-trained model for English.
2. For a word like `apple`, we get the word's vector (embedding) using spaCy's `doc.vector`.
3. We print the embedding vector and demonstrate how to compute the similarity (in this case, using the dot product) between two words (`apple` and `banana`).

Notes:

- The `en_core_web_sm` model uses relatively simple embeddings, but larger models (like `en_core_web_md` or `en_core_web_lg`) provide richer embeddings with more detailed word representations.
- If you want to use more advanced embeddings, you can download larger models or use pre-trained models like GloVe or Word2Vec.

Vectors & Numpy Arrays

Dot Product

MAI 554

DEEP LEARNING FOR LANGUAGE MODELING

LECTURE 3

EMBEDDINGS AND VECTOR REPRESENTATION

Dot Product

- **Linear Algebra Concept:**
  - **Dot Product:** The dot product of two vectors, feature vector $\mathbf{x}$ and weight vector $\mathbf{w}$, is calculated as $\mathbf{x} \cdot \mathbf{w} = \sum_{i=1}^{n} x_i w_i$. This operation can be seen as a linear transformation of the features by the weights.
    - **Properties:**
      - **Commutative:** $\mathbf{x} \cdot \mathbf{w} = \mathbf{w} \cdot \mathbf{x}$
      - **Distributive over addition**
      - **Scalar multiplication compatibility**

NumPy Implementation:

- **Computing Dot Product:**

```python
import numpy as np
x = np.array([2, 4, 6])  # Feature vector
w = np.array([1, 0, 1])  # Weight vector
dot_product = np.dot(x, w)  # Dot product using np.dot
dot_product_operator = x @ w  # Dot product using @ operator
```

Dot Product

- **Definition:**
  - The dot product of two vectors $\mathbf{x}$ and $\mathbf{y}$ is a scalar that represents the sum of the products of their corresponding components.
- **Compact Math Form:**
  - $\mathbf{x} \cdot \mathbf{y} = \sum_{i=1}^{n} x_i y_i$
- **Expanded Math Form:**
  - Given vectors $\mathbf{x} = [x_1, x_2, x_3]$ and $\mathbf{y} = [y_1, y_2, y_3]$, the dot product can be expressed as:

$$\mathbf{x} \cdot \mathbf{y} = \begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \cdot \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = x_1 \cdot y_1 + x_2 \cdot y_2 + x_3 \cdot y_3$$

- **Numerical Example:**
  - For vectors $\mathbf{x} = [2, 3, 4]$ and $\mathbf{y} = [1, 0, -1]$:

$$\mathbf{x} \cdot \mathbf{y} = 2 \cdot 1 + 3 \cdot 0 + 4 \cdot (-1) = 2 + 0 - 4 = -2$$

- **Purpose:**
  - The dot product quantifies how much one vector extends in the direction of another, thus providing a measure of alignment between the vectors. It is widely used to measure vector similarity in various fields of mathematics and physics.

```python
import numpy as np

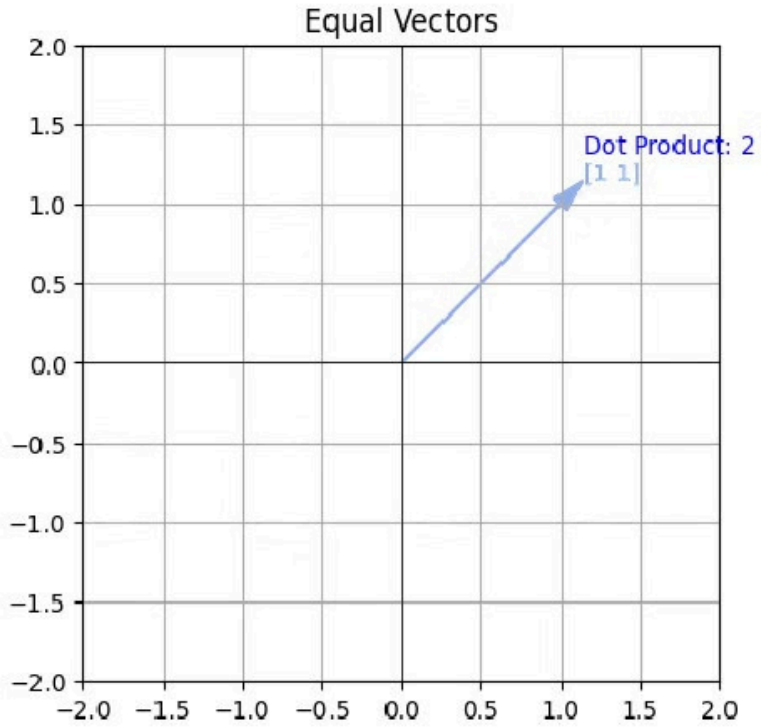
# Define the vectors
x = np.array([2, 3, 4])
y = np.array([1, 0, -1])

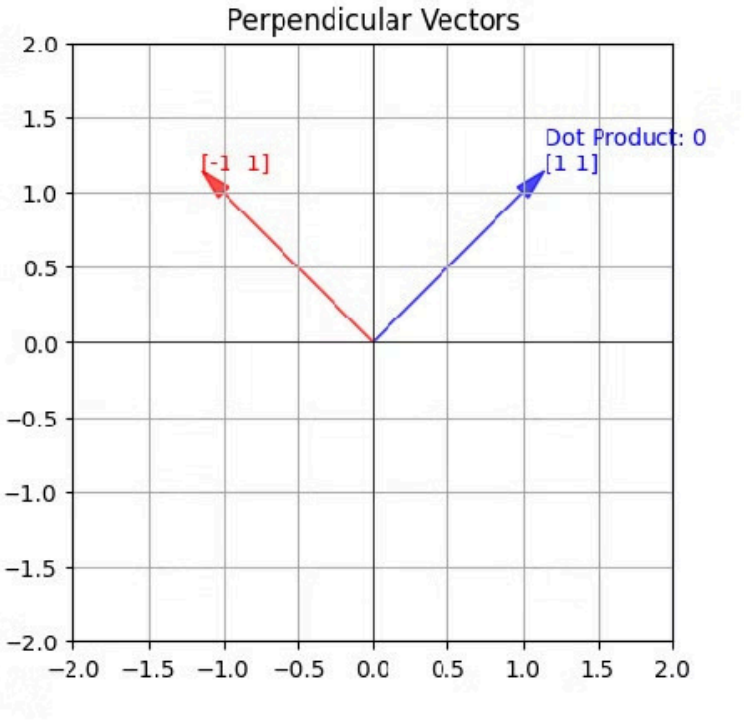
# Compute the dot product using NumPy
dot_product = np.dot(x, y)

# Print the results
print("Vectors x and y:")
print("x =", x)
print("y =", y)
print("Dot Product (x · y) =", dot_product)
```

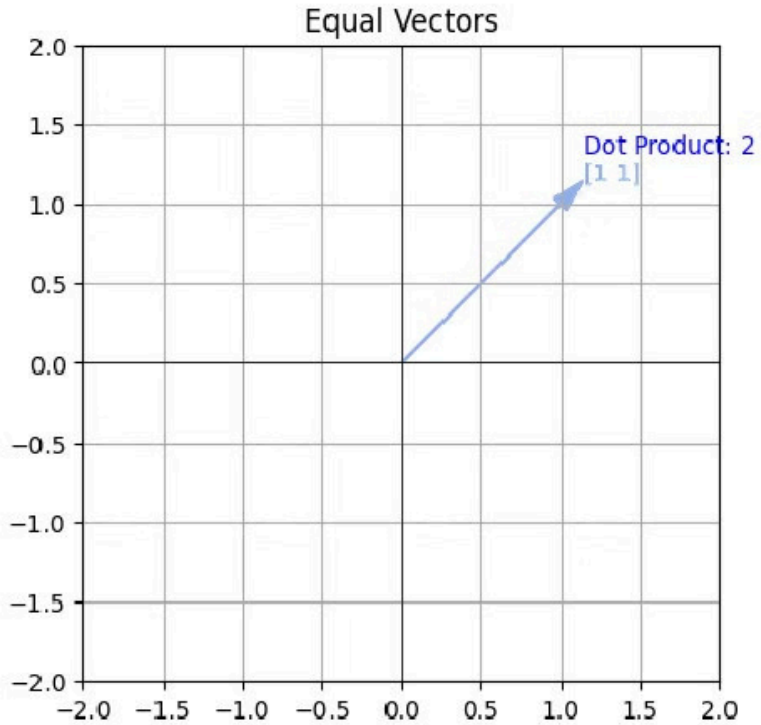
```
Vectors x and y:
x = [2 3 4]
y = [ 1  0 -1]
Dot Product (x · y) = -2
```

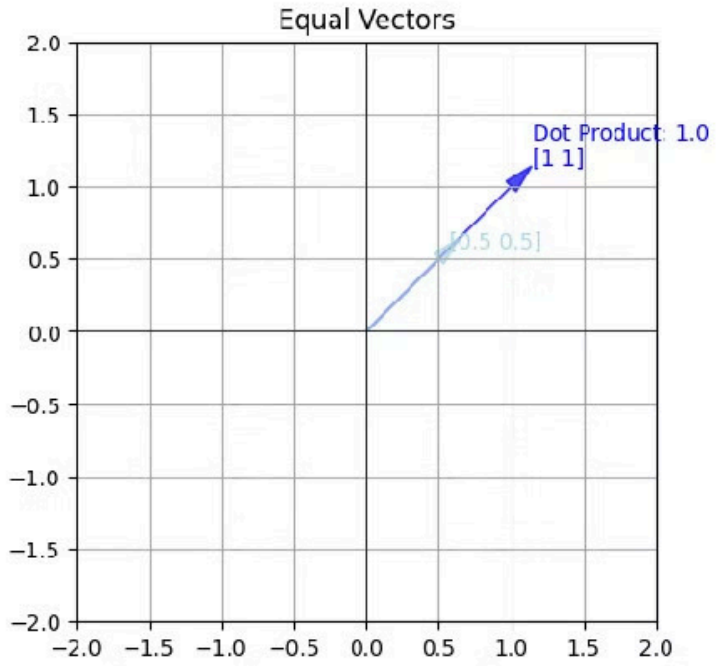
Why do We need Dot Product?



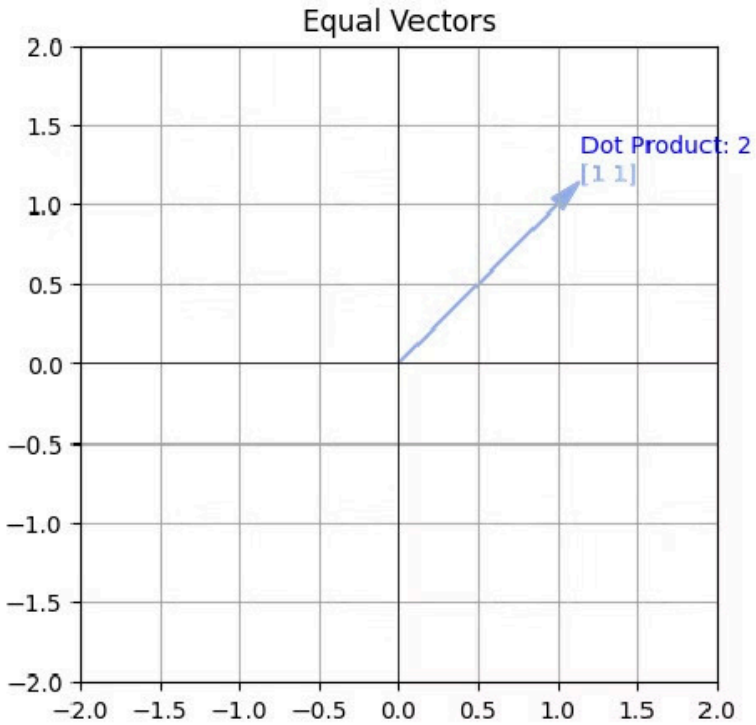


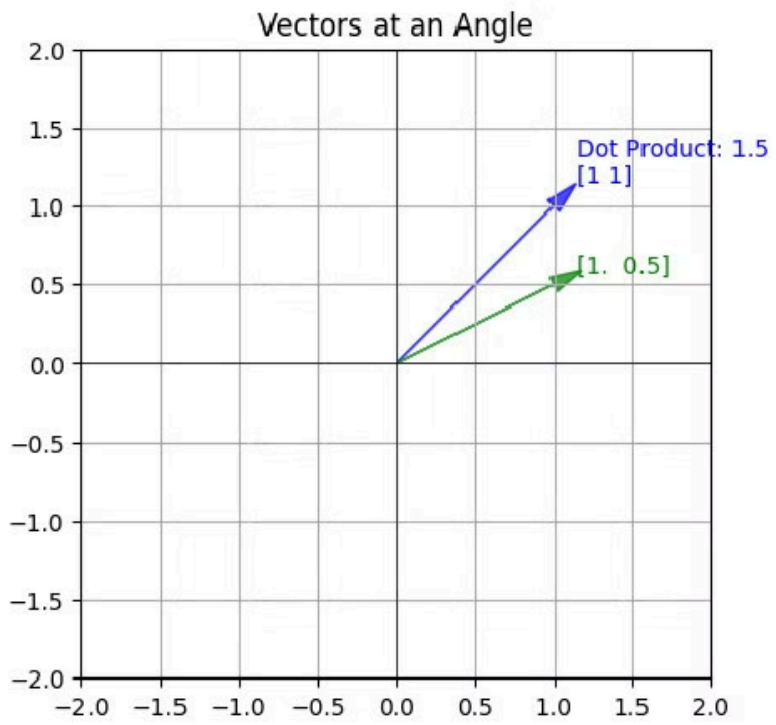
Why do We need Dot Product?



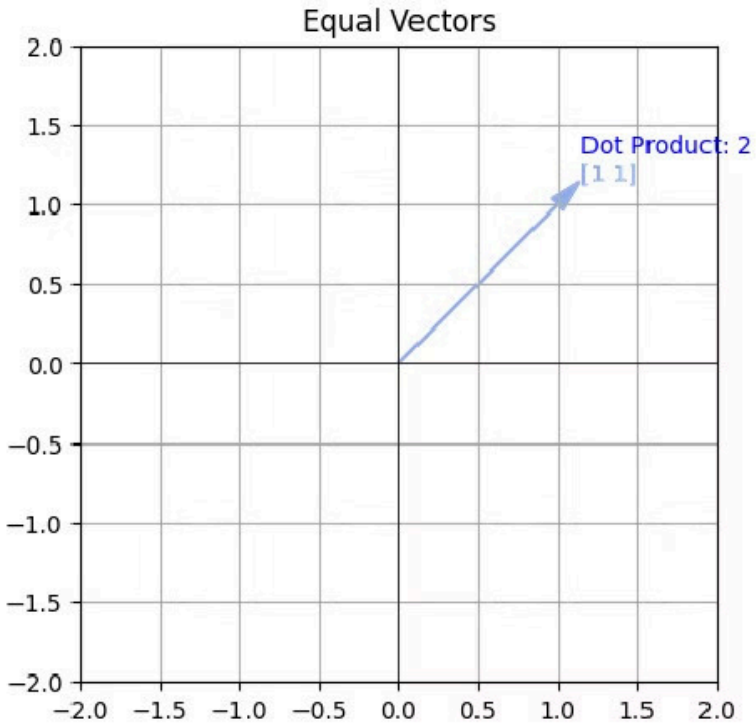


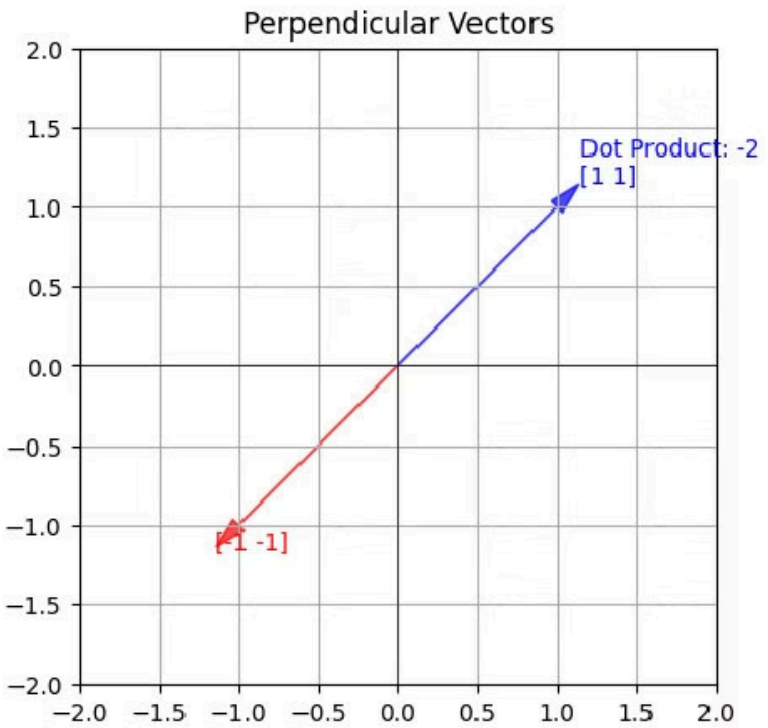
Why do We need Dot Product?





Why do We need Dot Product?





Why do We need Dot Product?

```
Dot product (similarity) between 'apple' and 'banana': 911.4376831054688
Dot product (similarity) between 'apple' and 'lion': 161.2523651123047
Embedding Shape: (300,)
```

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for better embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors
for word, vector in embeddings.items():
    print(f"Embedding vector for '{word}':")
    print(vector, "\n")

# Calculate the dot product between 'apple' and 'banana', and 'apple' and 'lion'
similarity_apple_banana = np.dot(embeddings['apple'], embeddings['banana'])
similarity_apple_lion = np.dot(embeddings['apple'], embeddings['lion'])

# Print the results
print(f"Dot product (similarity) between 'apple' and 'banana': {similarity_apple_banana}")
print(f"Dot product (similarity) between 'apple' and 'lion': {similarity_apple_lion}")

```

Why do We need Dot Product?

```
Dot product (similarity) between 'apple' and 'banana': 39.10390090942383
Dot product (similarity) between 'apple' and 'lion': 35.11163330078125
Embedding Shape: (96,)
```

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for better embeddings
nlp = spacy.load("en_core_web_sm")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors
for word, vector in embeddings.items():
    print(f"Embedding vector for '{word}':")
    print(vector, "\n")

# Calculate the dot product between 'apple' and 'banana', and 'apple' and 'lion'
similarity_apple_banana = np.dot(embeddings['apple'], embeddings['banana'])
similarity_apple_lion = np.dot(embeddings['apple'], embeddings['lion'])

# Print the results
print(f"Dot product (similarity) between 'apple' and 'banana': {similarity_apple_banana}")
print(f"Dot product (similarity) between 'apple' and 'lion': {similarity_apple_lion}")
print(f"Embedding Shape: {embeddings['apple'].shape}")

```

Vector Normalization

MAI 554

DEEP LEARNING FOR LANGUAGE MODELING

LECTURE 3

EMBEDDINGS AND VECTOR REPRESENTATION

Vector Normalization (L2)

- **Definition**:
  - Vector normalization is the process of scaling a vector to have a unit length, i.e., a magnitude of 1. This is accomplished by dividing each component of the vector by its norm (magnitude).
- **Compact Math Form**:
  - The normalized vector $\hat{\mathbf{v}}$ of a vector $\mathbf{v}$ is given by:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

- **Expanded Math Form**:
  - Given a vector $\mathbf{v} = [v_1, v_2, \ldots, v_n]$, its normalized form $\hat{\mathbf{v}}$ is:

$$\hat{\mathbf{v}} = \left[\frac{v_1}{\sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}}, \frac{v_2}{\sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}}, \ldots, \frac{v_n}{\sqrt{v_1^2 + v_2^2 + \ldots + v_n^2}} \right]$$

- **Purpose**:
  - **Similarity of Two Vectors**:
    - Normalization is crucial for methods like cosine similarity, where the similarity between two vectors is measured based on the angle between them, rather than their magnitude. By normalizing vectors, we ensure that the similarity measure reflects directional similarity, independent of vector length.

Vector Normalization - Example

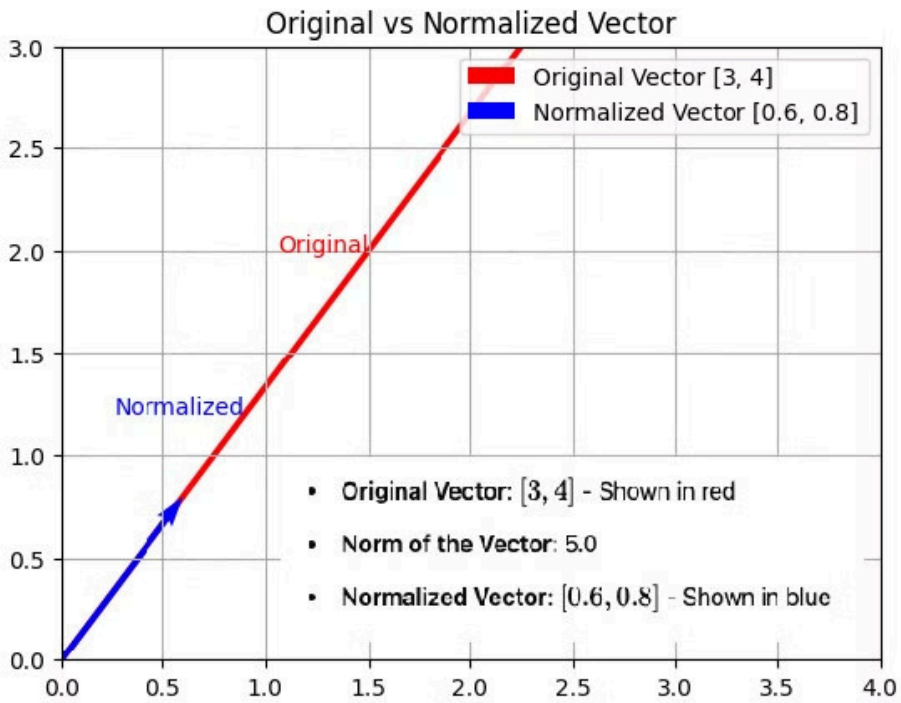
- **Expanded Math Form**:
  - For vector $\mathbf{v} = [3, 4]$:

$$\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = 5$$

$$\hat{\mathbf{v}} = \left[\frac{3}{5}, \frac{4}{5}\right]$$

- **Numerical Example**:
  - Normalize $\mathbf{v} = [3, 4]$.
    - Norm $\|\mathbf{v}\| = 5$.
    - Normalized Vector $\hat{\mathbf{v}} = [0.6, 0.8]$.
- **NumPy Implementation**:
  -

```python
import numpy as np
v = np.array([3, 4])
norm_v = np.linalg.norm(v)
normalized_v = v / norm_v
print("Original Vector:", v)
print("Norm of v:", norm_v)
print("Normalized Vector:", normalized_v)
```



L1-Normalization

L1 Norm (Manhattan Distance):

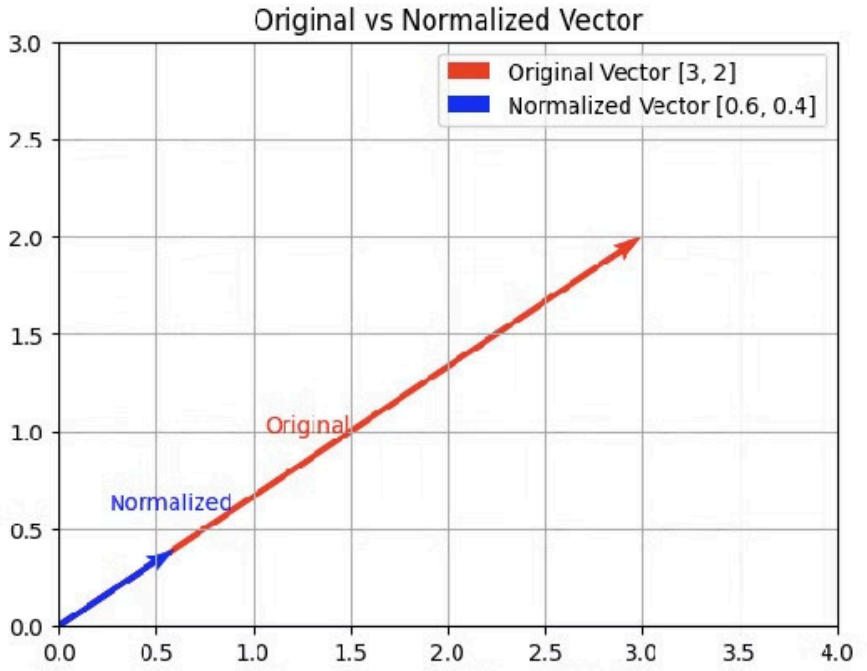
- **Compact Form:**

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i|$$

- **Expanded Form:**
  - For vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$:

$$\|\mathbf{x}\|_1 = |x_1| + |x_2| + \ldots + |x_n|$$

- **Purpose**: Measures the sum of the absolute values of the components. It is useful in scenarios where differences of any size are equally important.



L2-Normalization

L2 Norm (Euclidean Distance):

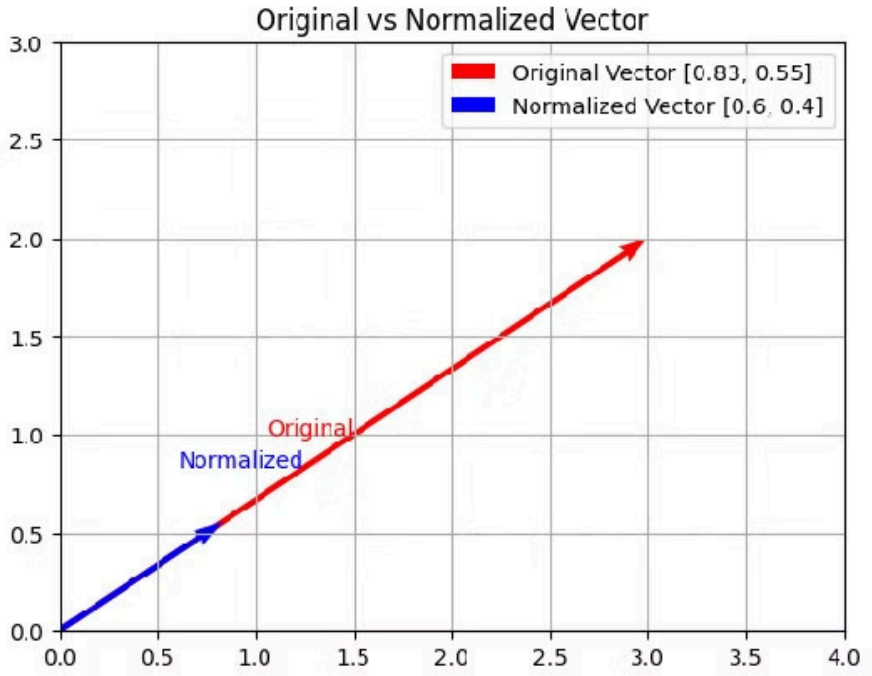
- **Compact Form:**

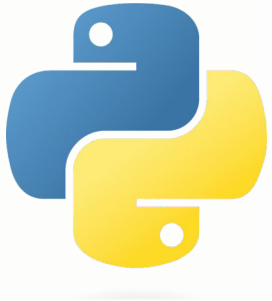
$$||\mathbf{x}||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

- **Expanded Form:**
  - For vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]$:

$$||\mathbf{x}||_2 = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2}$$

- **Purpose:** Measures the straight-line distance from the origin to the point in n-dimensional space, which is useful for determining actual distances between points.





Vectors & Numpy Arrays

Vector Normalization

CS316

INTRODUCTION TO AI AND DATA SCIENCE

CHAPTER 3

PANDAS AND NUMPY

LECTURE 1

PANDAS

Normalization in NLP

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors and calculate their norms (L2 norm)
for word, vector in embeddings.items():
    # Compute the L2 norm (Euclidean norm) of the word embedding
    norm = np.linalg.norm(vector)
    print(f"Vector norm (L2) for '{word}': {norm}")

```

```
Vector norm (L2) for 'apple': 43.36647415161133
Vector norm (L2) for 'banana': 31.6203556060791
Vector norm (L2) for 'lion': 55.14573287963867
```


```
Vector norm (L2) for 'apple': 43.36647415161133
Vector norm (L2) for 'banana': 31.6203556060791
Vector norm (L2) for 'lion': 55.14573287963867
```

Normalization in NLP

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Function to normalize a vector
def normalize_vector(vector):
    norm = np.linalg.norm(vector)  # Calculate the L2 norm
    if norm == 0:  # To prevent division by zero
        return vector
    return vector / norm

# Normalize the embeddings and show the result
for word, vector in embeddings.items():
    normalized_vector = normalize_vector(vector)

    # Show original and normalized vector norms
    print(f"Original L2 norm for '{word}': {np.linalg.norm(vector)}")
    print(f"Normalized L2 norm for '{word}': {np.linalg.norm(normalized_vector)}")
    print(f"Normalized vector for '{word}':\n{normalized_vector}\n")

```

```
1.0
Original L2 norm for 'apple': 43.36647415161133
Normalized L2 norm for 'apple': 1.0
Normalized vector for 'apple':
[-0.02325298 -0.0468288  -0.0148006   0.06209405  0.00732617 -0.06148067
 -0.08617717  0.12616658 -0.11933412  0.01198114  0.12657934 -0.04823542
 -0.05749142  0.05338686  0.00906207 -0.05597101  0.07096383 -0.06115323
  0.05843454 -0.11089443 -0.04526769  0.11309428 -0.05865822 -0.03650285
 -0.00719639 -0.05750064 -0.05594414  0.00798733  0.01064832  0.02833064
 -0.00483484 -0.03240522 -0.0555429  -0.05133609  0.0328249  -0.04312087
  0.05970511  0.07801418  0.06552527 -0.03466042  0.02447513  0.03105394
  0.01761545  0.02147581  0.04973427  0.05345143  0.03137677 -0.00476566
  0.06601644  0.01765949  0.0379579   0.01013041  0.0476451  -0.05680425
 -0.06277199  0.0271869   0.00095576  0.05129308  0.04127843 -0.01273749
  0.06190266  0.08541851 -0.01359553 -0.00046061 -0.03268423 -0.0099736
 -0.04702942 -0.12144865  0.01810177  0.00215099 -0.01777687  0.04564355
 -0.02931758 -0.01038959  0.05871356  0.0341093  -0.10125333 -0.06029681
  0.05877351 -0.03483567  0.00474906  0.0393207  -0.00729227 -0.01800561
  0.02562809  0.10181367  0.12546559  0.10126255 -0.06089958  0.02357812
 -0.00602862  0.07747921  0.06238921  0.04965356  0.0086131   0.00793077
  0.11659697 -0.06637616 -0.0374921  -0.07797268 -0.00067771 -0.03695481
```



Cosine Similarity



Cosine Similarity of Two Vectors

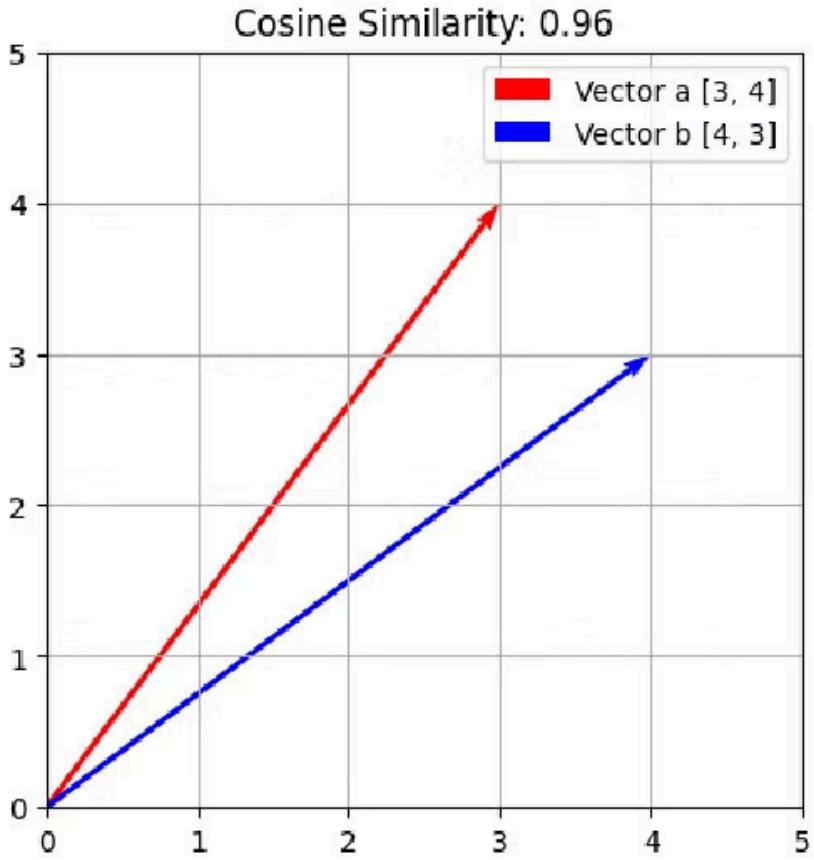
- **Definition:**
  - Cosine similarity is a measure of similarity between two non-zero vectors within an inner product space that measures the cosine of the angle between them.
- **Math Equation:**
  - **Compact Form:**

$$\text{Cosine Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$

  - **Expanded Form:**
    - Given vectors $\mathbf{x} = [x_1, x_2, \ldots, x_n]$ and $\mathbf{y} = [y_1, y_2, \ldots, y_n]$:

$$\text{Cosine Similarity} = \frac{x_1 y_1 + x_2 y_2 + \ldots + x_n y_n}{\sqrt{x_1^2 + x_2^2 + \ldots + x_n^2} \cdot \sqrt{y_1^2 + y_2^2 + \ldots + y_n^2}}$$

- **Purpose:**
  - **Similarity Measurement:**
    - Cosine similarity is widely used in various applications such as text analysis, where it helps in identifying the similarity between documents. It is especially useful in high-dimensional positive spaces like text data in TF-IDF representation.
    - In machine learning, it helps in clustering and classification by measuring how similar the data objects are, irrespective of their magnitude.



Cosine Similarity of Two Vectors

```python
import numpy as np
import matplotlib.pyplot as plt

# Define the vectors
a = np.array([3, 4])
b = np.array([4, 3])

# Calculate the dot product
dot_product = np.dot(a, b)

# Calculate the norms of the vectors
norm_a = np.linalg.norm(a)
norm_b = np.linalg.norm(b)

# Calculate the cosine similarity
cosine_similarity = dot_product / (norm_a * norm_b)

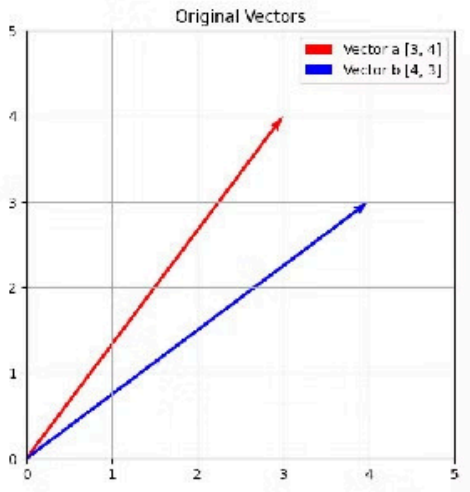
print("Cosine Similarity:", cosine_similarity)
```

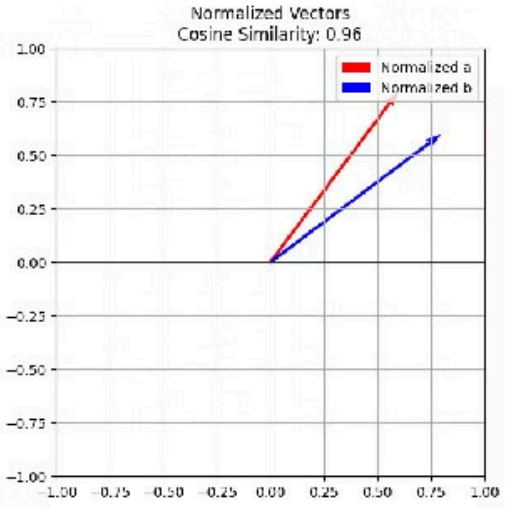
Numerical Example:

Consider two vectors in a 2D space:

- $\mathbf{a} = [3, 4]$
- $\mathbf{b} = [4, 3]$

$$\text{Cosine Similarity}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$$





Normalization in NLP

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Display the embedding vectors and calculate their norms (L2 norm)
for word, vector in embeddings.items():
    # Compute the L2 norm (Euclidean norm) of the word embedding
    norm = np.linalg.norm(vector)
    print(f"Vector norm (L2) for '{word}': {norm}")

```

```
Vector norm (L2) for 'apple': 43.36647415161133
Vector norm (L2) for 'banana': 31.6203556060791
Vector norm (L2) for 'lion': 55.14573287963867
```


```
Vector norm (L2) for 'apple': 43.36647415161133
Vector norm (L2) for 'banana': 31.6203556060791
Vector norm (L2) for 'lion': 55.14573287963867
```

Normalization in NLP

```python
import spacy
import numpy as np

# Load the medium English model in spaCy for word embeddings
nlp = spacy.load("en_core_web_md")

# Define the words
words = ["apple", "banana", "lion"]

# Get the embeddings for the words
embeddings = {word: nlp(word).vector for word in words}

# Function to normalize a vector
def normalize_vector(vector):
    norm = np.linalg.norm(vector)  # Calculate the L2 norm
    if norm == 0:  # To prevent division by zero
        return vector
    return vector / norm

# Normalize the embeddings and show the result
for word, vector in embeddings.items():
    normalized_vector = normalize_vector(vector)

    # Show original and normalized vector norms
    print(f"Original L2 norm for '{word}': {np.linalg.norm(vector)}")
    print(f"Normalized L2 norm for '{word}': {np.linalg.norm(normalized_vector)}")
    print(f"Normalized vector for '{word}':\n{normalized_vector}\n")

```

```
1.0
Original L2 norm for 'apple': 43.36647415161133
Normalized L2 norm for 'apple': 1.0
Normalized vector for 'apple':
[-0.02325298 -0.0468288  -0.0148006   0.06209405  0.00732617 -0.06148067
 -0.08617717  0.12616658 -0.11933412  0.01198114  0.12657934 -0.04823542
 -0.05749142  0.05338686  0.00906207 -0.05597181  0.07896883 -0.06115323
  0.05843454 -0.11089443 -0.04526769  0.11309428 -0.05865822 -0.03650285
 -0.08719639 -0.05750064 -0.05594414  0.08798733  0.01064832  0.02833064
 -0.00483484 -0.03240522 -0.0555429  -0.05133689  0.0328249  -0.04312087
  0.05970511  0.07801418  0.06552527 -0.03466042  0.02447513  0.03105394
  0.01761545  0.02147581  0.04973427  0.05345143  0.03137677 -0.00476566
  0.06601644  0.01765949  0.0379579   0.01013041  0.0476451  -0.05680425
 -0.06277199  0.0271869   0.00095576  0.05129308  0.04127843 -0.01273749
  0.06190266  0.08541851 -0.01359553 -0.00046061 -0.03268423 -0.0099736
 -0.04702942 -0.12144865  0.01810177  0.00215099 -0.01777687  0.04564355
 -0.02931758 -0.01038959  0.05871356  0.0341093  -0.10125333 -0.06029081
  0.05877351 -0.03483567  0.00474906  0.0393207  -0.00729227 -0.01800561
  0.02562809  0.10181367  0.12546559  0.10126255 -0.06089958  0.02357812
 -0.00602862  0.07747921  0.06238921 -0.04965356  0.0086131  -0.00793977
  0.11659697 -0.06637616 -0.0374921  -0.07797268 -0.00067771 -0.03695481
```